

Research Proposal

Significant Productivity Gains through Programming with Large Language Models

Prepared by:

Muhammad Zain | ID: F2023266257

Muhammad Umar | ID: F20232661022

1. Project Title

Evaluating the Impact of Large Language Models on Developer Productivity: A Comparative Study of Auto-complete and Conversational Interfaces

2. Executive Summary

The rapid emergence of Large Language Models (LLMs) such as GPT-3 and Codex is transforming software development. Tools like GitHub Copilot and ChatGPT can rapidly generate code from natural language prompts, promising immense productivity benefits. However, empirical understanding of how different AI interaction paradigms affect developers holistically is limited.

This proposal outlines a within-subjects user study (N=24) to evaluate programmer productivity across three conditions: a traditional web-browser baseline, an auto-complete AI interface (GitHub Copilot), and a conversational AI interface (GPT-3). Using the multi-dimensional SPACE framework (Satisfaction, Performance, Activity, Communication, Efficiency), this research aims to quantify productivity gains, analyze distinctive usage patterns, and provide actionable insights for the future design of human-centered AI coding assistants.

3. Background and Problem Statement

Modern Machine Learning (ML) systems have evolved beyond simple code completion. LLM-backed tools can now generate complex software components from informal specifications. Despite rapid adoption, their development is predominantly technology-driven. Current research

presents a heterogeneous picture: some studies report massive speed improvements, while others note severe debugging challenges caused by overreliance on AI.

Problem Statement: There is a lack of systematic, human-centered analysis regarding how different forms of AI presentation (auto-complete vs. conversational) impact the software developer's holistic productivity, behavior, and satisfaction.

4. Research Objectives and Questions

This research seeks to achieve the following overarching objectives:

1. **Compare Modalities:** To evaluate developer productivity when using an auto-complete AI assistant versus a conversational AI assistant, compared against a traditional internet-search baseline.
2. **Apply the SPACE Framework:** To adopt a comprehensive assessment model measuring Satisfaction, Performance, Activity, Communication, and Efficiency, moving beyond rudimentary "Lines of Code" metrics.
3. **Identify Usage Patterns:** To observe and document the distinct strategies and behaviors developers employ when integrating varying AI tools into their workflow.

To fulfill these objectives, the study will address the following specific **Research Questions (RQs)**:

- **RQ1:** How do auto-complete and conversational AI interfaces differ in their impact on the *Efficiency* and *Performance* of developers compared to a web-search baseline?
- **RQ2:** How does the modality of the AI assistant alter the *Activity* and *Communication* patterns (e.g., snippet size, acceptance rate) during software development?
- **RQ3:** What is the effect of these different AI interfaces on the subjective *Satisfaction* and cognitive load of the developers?

5. Literature Review

The shift from syntax-based code completion to semantic, AI-driven generation marks a paradigm shift in software engineering.

- **AI-Assisted Programming:** Tools based on Transformer architectures (e.g., GPT, PaLM, CodeBERT) have made code generation highly accessible (Chen et al., 2021). Previous

evaluations largely focused on the functional correctness of the generated code rather than the human factors involved in its creation.

- **Productivity Measurement:** Traditional metrics like Lines of Code (LoC) or Function Points lack the contextual nuance required for modern development evaluation. The SPACE framework (Forsgren et al., 2021) offers a robust alternative by integrating human-centric dimensions.
- **Current Evidence:** Peng et al. (2023) demonstrated a 55.8% task completion time improvement using GitHub Copilot. Conversely, Vaithilingam et al. (2022) found no significant time reduction, citing increased debugging time. This discrepancy highlights the necessity of evaluating the interaction paradigms themselves.

6. Proposed Methodology

6.1 Study Design

A within-subjects experimental design will be employed. Each participant will complete three distinct programming tasks under three different environmental conditions:

1. **Baseline:** Integrated Development Environment (IDE) with standard web browser access.
2. **Auto-complete:** IDE enhanced with an inline AI auto-completion tool (GitHub Copilot).
3. **Conversational:** IDE integrated with an AI chatbot interface (GPT-3).

6.2 Participants

We aim to recruit 24 participants spanning varying levels of programming expertise (novice to professional) and AI familiarity to ensure ecological validity. Inclusion criteria require basic proficiency in Python.

6.3 Tasks and Apparatus

Participants will solve three equivalent Python programming tasks (Text Analysis, CSV Transformation, PDF Editing). The combinations of task and condition will be counterbalanced using a Latin Square design to mitigate order effects. The environment will be controlled via a standardized Virtual Machine running Visual Studio Code.

6.4 Data Collection and Metrics (The SPACE Framework)

- **Satisfaction:** Evaluated via AttrakDiff questionnaires and qualitative post-task surveys.

- **Performance:** Code quality measured objectively through unit testing (functional correctness) and static analysis (Maintainability Index).
- **Activity:** Keystroke logging normalized by time, analyzing characters typed per minute and the ratio of human-written vs. AI-generated code.
- **Communication:** Analysis of the source (Browser vs. AI), frequency, and average size of accepted code snippets.
- **Efficiency:** Measured by the number of specific task requirements successfully implemented per minute.

6.5 Data Analysis Plan

Quantitative data will be tested for normality using the Shapiro-Wilk test. Depending on the distribution, statistical significance will be determined using repeated measures ANOVAs (with post-hoc t-tests) or Friedman tests (with post-hoc Wilcoxon signed-rank tests), applying Bonferroni corrections to adjust for multiple comparisons. Qualitative survey feedback will be analyzed through thematic coding to extract recurring sentiments regarding user satisfaction and cognitive load.

7. Limitations and Risk Management

While robust, the study design has inherent limitations that will be proactively managed:

- **Sample Size & Demographics:** A sample of 24 participants may limit broad generalizability. We will mitigate this by ensuring a diverse recruitment pool (e.g., local software houses in IT hubs like Arfa Software Technology Park, and universities such as FAST-NUCES or PUCIT) with varying levels of prior AI exposure.
- **Language Specificity:** Tasks are limited to Python. While Python is highly representative of general-purpose programming, future work will be needed to validate findings across other languages (e.g., C++, Java).
- **Constrained Tasks:** The 15-minute timebox per task limits open-ended creativity. However, this constraint ensures consistency and comparability across the three test conditions.

8. Expected Outcomes, Significance, and Dissemination

We anticipate that AI assistance will significantly enhance task completion speed and overall efficiency compared to the baseline. Furthermore, we hypothesize that the modality of the AI will shape developers' behavior: auto-complete tools may foster a rapid, iterative approach with short

snippets, whereas conversational tools may be utilized for generating broader, architectural code blocks.

The findings will contribute a holistic understanding of AI's current impact on software engineering, bridging the gap between technological capabilities and human-computer interaction needs, ultimately guiding the design of future IDEs.

Dissemination Plan: Results will be compiled into a peer-reviewed manuscript targeted for top-tier international conferences (e.g., ACM CHI, ICSE) as well as prominent national forums like IEEE FIT (Frontiers of Information Technology) in Pakistan. Additionally, all anonymized datasets and analysis scripts will be open-sourced to promote reproducibility.

9. Proposed Work Plan and Timeline

Phase	Activity	Duration
Phase 1	Finalization of task design, tutorial creation, and Virtual Machine configuration.	Weeks 1-2
Phase 2	Pilot testing (N=3) and iteration of task complexity and experimental flow.	Week 3
Phase 3	Participant recruitment and execution of the main within-subjects study.	Weeks 4-6
Phase 4	Data extraction, statistical analysis (ANOVAs, Wilcoxon tests), and qualitative coding.	Weeks 7-8
Phase 5	Synthesis of findings and drafting of the final research paper/report.	Weeks 9-10

10. Budget Estimation

Item	Description	Estimated Cost (PKR)
Participant Compensation	24 participants @ PKR 1,500/hour for 2 hours	PKR 72,000

Item	Description	Estimated Cost (PKR)
Cloud Infrastructure	Virtual Machine hosting for standardized environments	PKR 20,000
Miscellaneous	Software licenses, API credits, contingency	PKR 15,000
Total		PKR 107,000

11. References

- Barke, S., et al. (2022). Grounded Copilot: How Programmers Interact with Code-Generating Models.
- Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code.
- Forsgren, N., et al. (2021). The SPACE of Developer Productivity: There's more to it than you think. ACM Queue.
- Peng, S., et al. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.
- Vaithilingam, P., et al. (2022). Expectation vs. Experience: Evaluating the Usability of Code Generation Tools.